

UCS503P - SOFTWARE ENGINEERING PROJECT

BLURGUARD

AI-Based Real-Time Sensitive Data Detection and Masking

Submitted by:

1024170440 Lakshita Dabra

1024170448 Rhythm Kapila

1024170452 Yashjot Kaur

Group: BE Third Year, CSE

Submitted to:

Dr. Nisha Thakur

Mid-Semester Evaluation

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	2
1.1	Background and Context	2
1.1.1	Problem Statement	2
1.2	Project Objectives	2
2	Methodology and Design	3
2.1	System Architecture	3
2.1.1	Technical Stack	3
2.2	Implementation Details	3
2.2.1	Frontend Interface	3
2.2.2	Detection and Masking	5
2.3	System Modeling and Project Diagrams	5
2.3.1	Activity Diagram	5
2.3.2	Sequence Diagram	6
2.3.3	Collaboration Diagram	8
2.3.4	Project Timeline	8
3	Progress Evaluation	9
3.1	Current Status and Validation	9
3.2	Testing	9
4	Conclusion	11
4.1	Summary of Work	11
4.2	Future Work	11
	Bibliography	12

Chapter 1

Introduction

1.1 Background and Context

Screenshots and screen-sharing sessions may contain sensitive information such as names, email addresses, phone numbers, identification numbers, passwords, API keys, and financial information.

Manual detection and masking of such information is time-consuming and may result in accidental data exposure.

BlurGuard is proposed as an AI-based privacy protection system that detects sensitive information from visual content and masks the detected regions before sharing.

The system combines OCR, image processing, pattern matching, computer vision, and risk assessment.

1.1.1 Problem Statement

The main problem is the accidental exposure of sensitive information through screenshots, application windows, documents, and screen-sharing sessions.

BlurGuard aims to automate this process by:

- detecting sensitive information,
- identifying its location,
- assessing its risk, and
- masking the sensitive region.

1.2 Project Objectives

The main objectives are:

1. Detect sensitive information from images and screenshots.
2. Extract text using OCR.
3. Identify structured information using pattern matching.
4. Classify detected information according to risk.
5. Mask sensitive regions automatically.
6. Provide a user interface for monitoring privacy protection.

Chapter 2

Methodology and Design

2.1 System Architecture

BlurGuard follows a modular pipeline:

Capture → Preprocess → OCR → Detect → Risk Assessment → Mask

The major modules are:

- Screen Capture
- Image Preprocessing
- OCR
- Sensitive Data Detection
- Risk Assessment
- Masking
- User Interface

2.1.1 Technical Stack

Table 2.1: Technical Stack

Technology	Purpose
Python	Backend processing
OpenCV	Image processing
Tesseract OCR	Text extraction
FastAPI	Backend API
React	User interface
TypeScript	Frontend development
Vite	Development environment
Git/GitHub	Version control

2.2 Implementation Details

2.2.1 Frontend Interface

The frontend provides the main interface for accessing and monitoring BlurGuard.

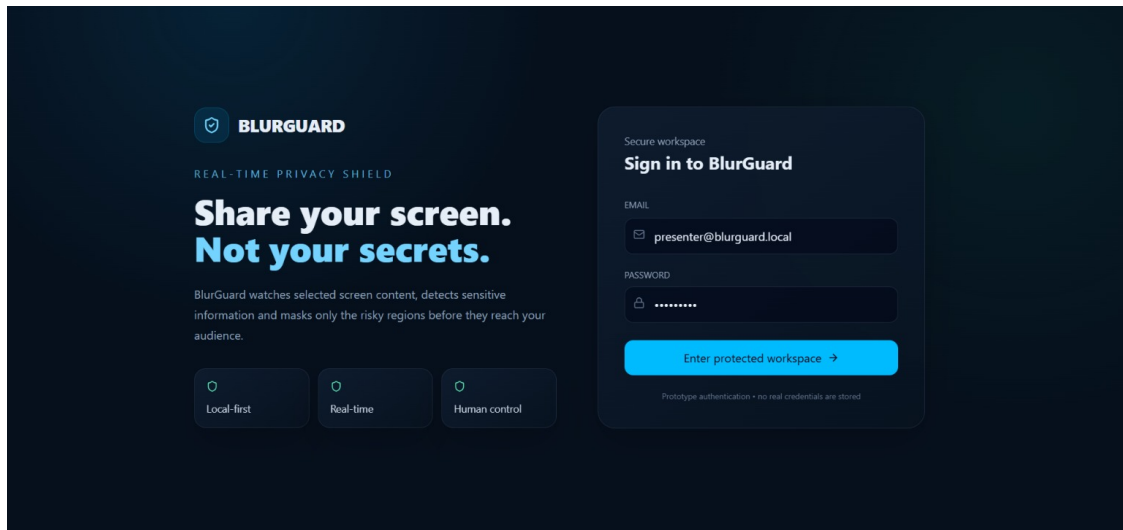


Figure 2.1: BlurGuard Secure Workspace

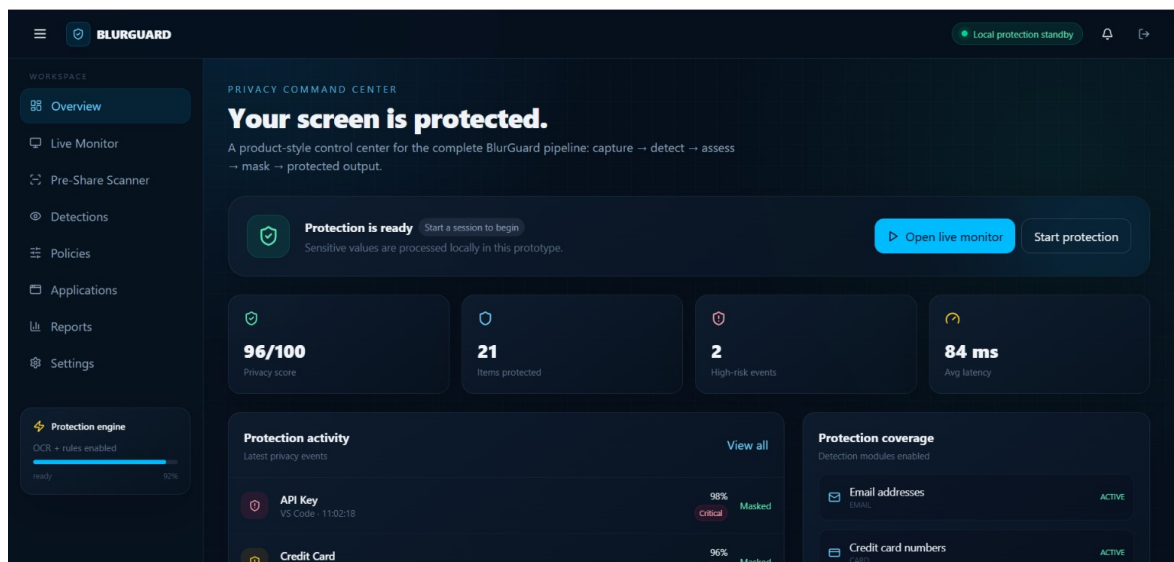


Figure 2.2: BlurGuard Privacy Command Center

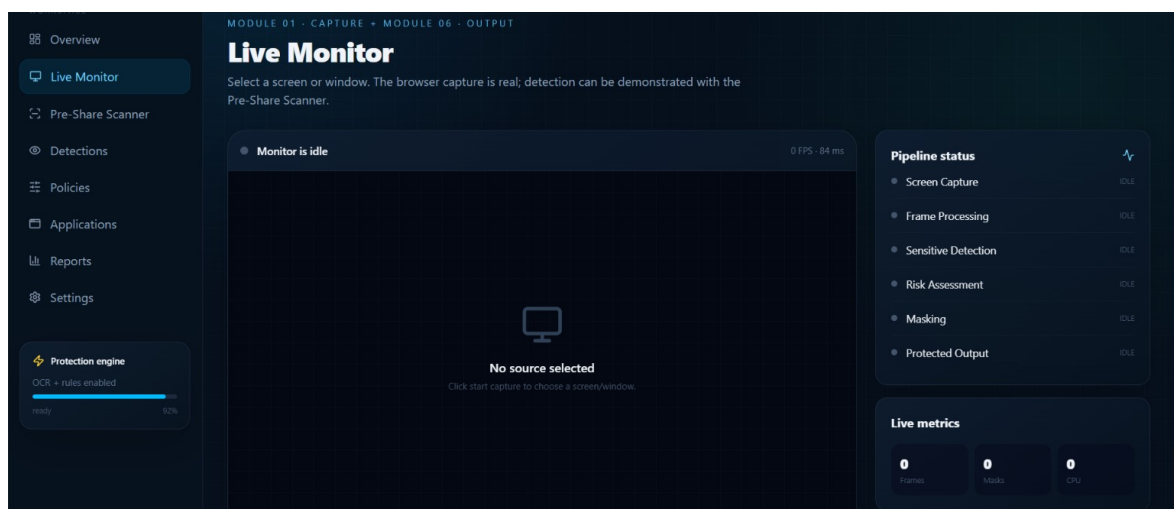


Figure 2.3: BlurGuard Live Monitoring Interface

2.2.2 Detection and Masking

The system uses OCR to extract visible text and pattern-based techniques to identify sensitive information.

Target categories include:

- Personal information
- Email addresses
- Phone numbers
- Financial information
- Passwords and credentials
- API keys and access tokens

Detected information is assigned a risk level and protected using blur, pixelation, solid masking, or redaction.

2.3 System Modeling and Project Diagrams

2.3.1 Activity Diagram

The activity diagram represents the flow from screen monitoring to detection and masking.

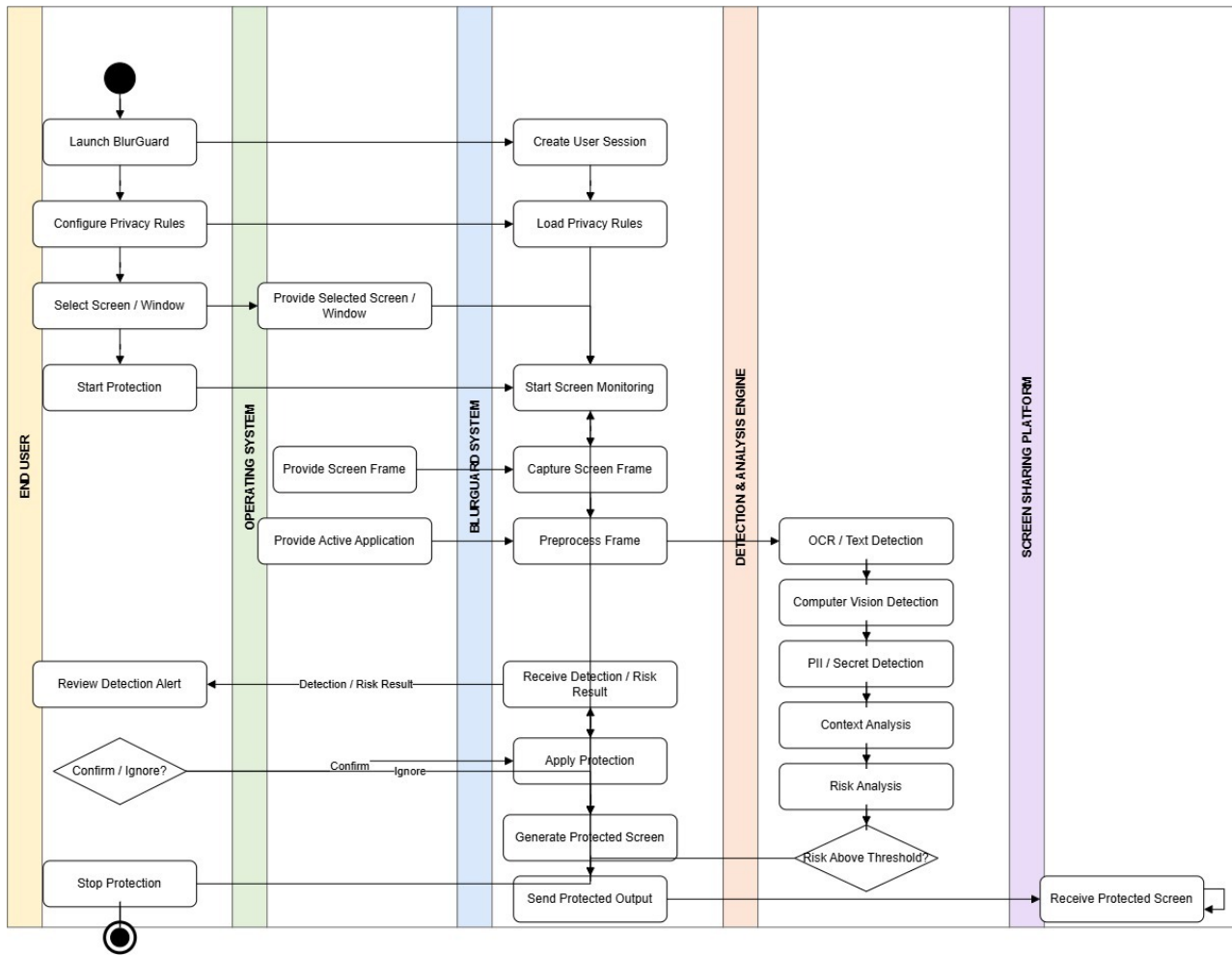
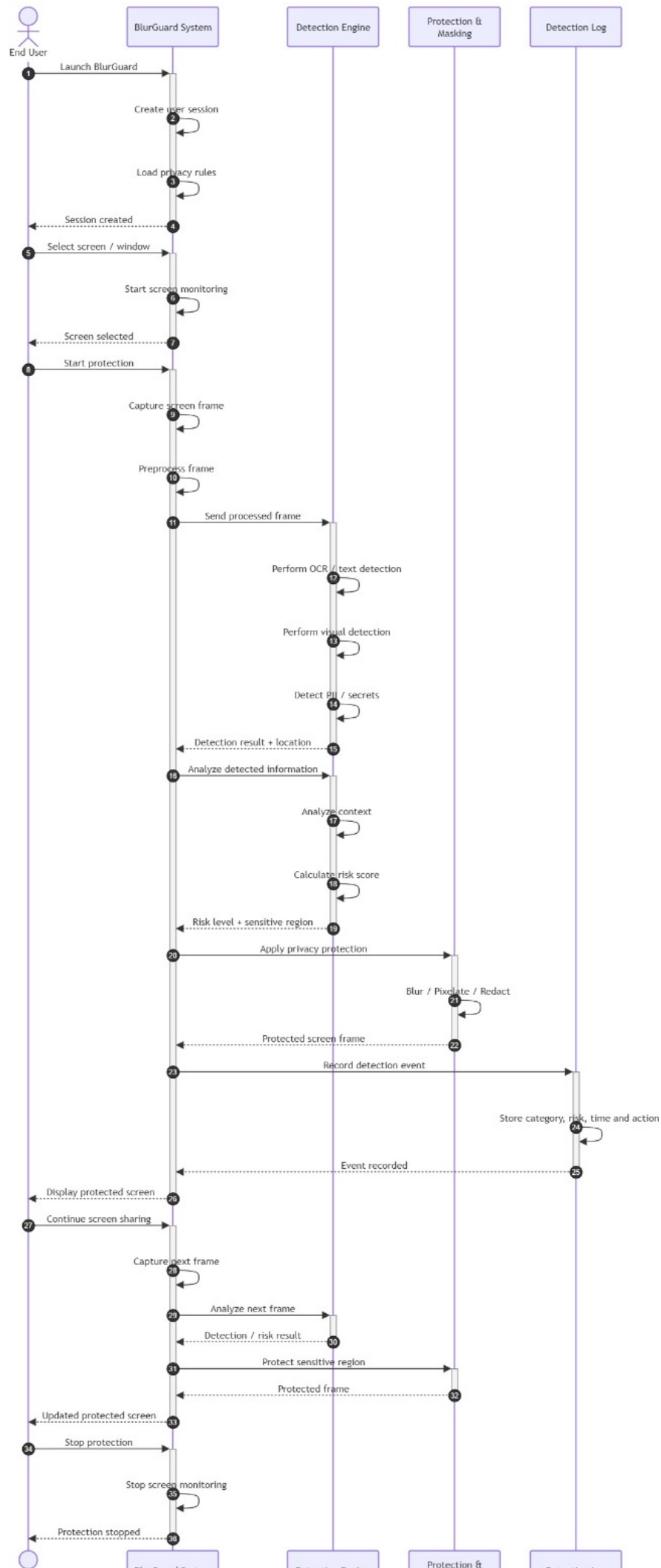


Figure 2.4: Activity and Swimlane Diagram

2.3.2 Sequence Diagram

The sequence diagram shows the chronological interaction between the user interface and processing modules.



2.3.3 Collaboration Diagram

The collaboration diagram represents communication between the major BlurGuard components.

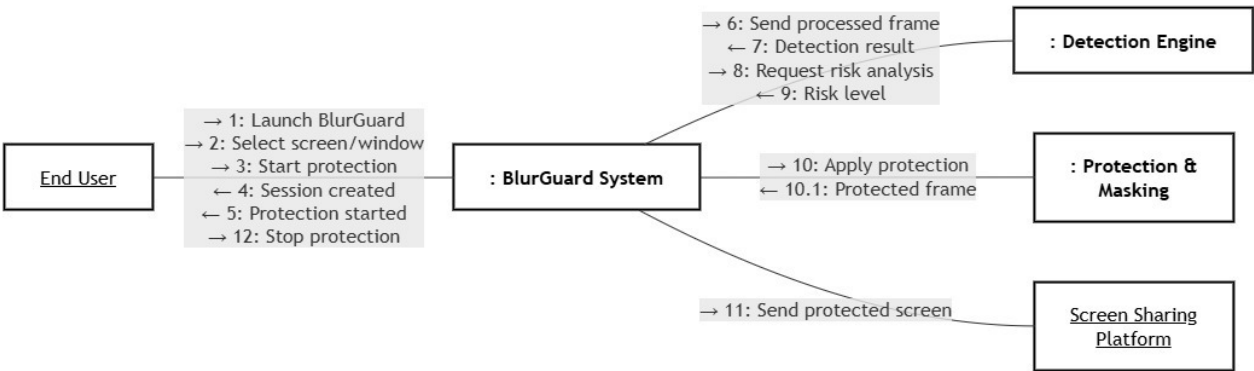


Figure 2.6: Collaboration Diagram of BlurGuard

2.3.4 Project Timeline

The Gantt chart represents the planned development activities.

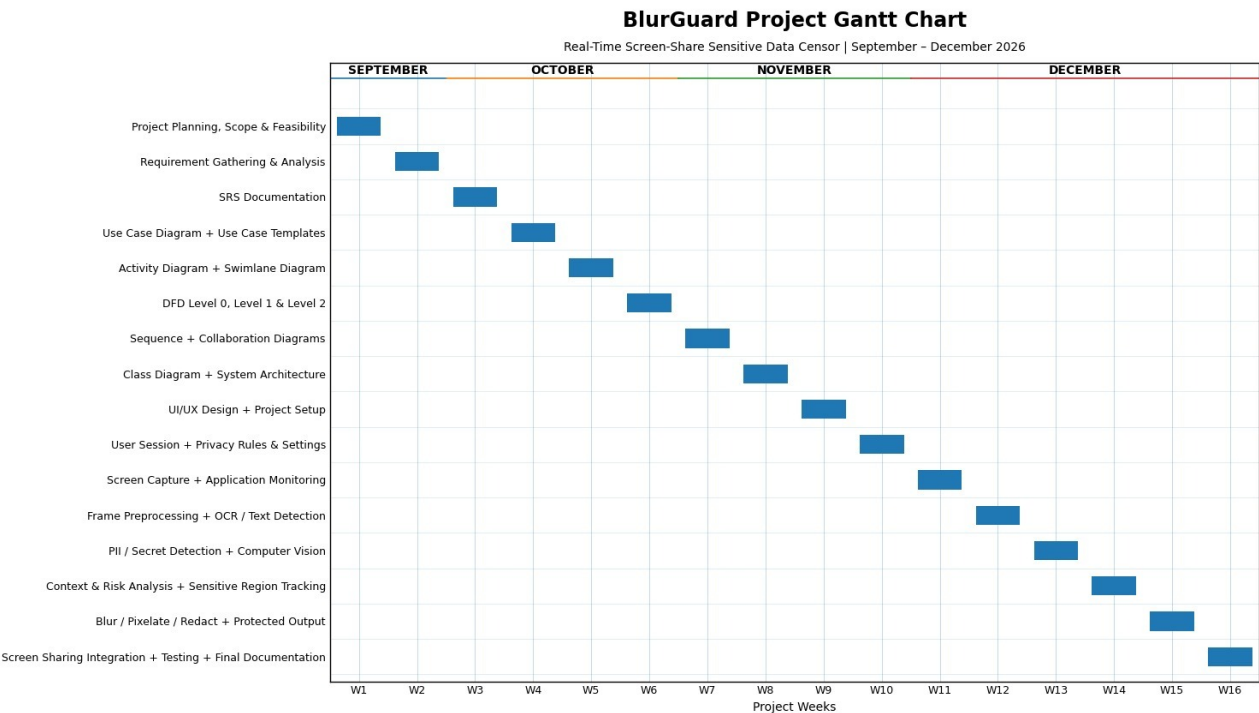


Figure 2.7: Project Scheduling and Timeline

Chapter 3

Progress Evaluation

3.1 Current Status and Validation

The project has completed the initial system architecture, interface design, UML modelling, and core privacy-protection workflow.

The current prototype includes:

- Secure Workspace
- Privacy Command Center
- Live Monitoring interface
- Detection workflow
- Risk classification
- Masking approach
- UML system models

3.2 Testing

Testing will focus on:

- OCR text extraction
- Sensitive-data detection
- Risk classification
- Region localisation
- Masking functionality
- Interface behaviour

The detection system can be evaluated using precision, recall, and F1-score after experimental testing.

$$Precision = \frac{TP}{TP + FP} \quad (3.1)$$

$$Recall = \frac{TP}{TP + FN} \quad (3.2)$$

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall} \quad (3.3)$$

No numerical performance values are reported until actual experiments are completed.

Chapter 4

Conclusion

4.1 Summary of Work

BlurGuard proposes an automated privacy-protection system for detecting and masking sensitive information in visual content.

The project combines image processing, OCR, pattern matching, risk assessment, and masking within a modular architecture.

The current work establishes the frontend interface, system workflow, UML models, and core detection approach.

4.2 Future Work

Future development will focus on:

- Real-time screen capture
- Improved OCR
- Context-aware detection
- Named Entity Recognition
- Additional sensitive-data categories
- Screen-sharing integration
- Confidence-based masking
- Performance optimisation

Bibliography

1. R. Smith, “An Overview of the Tesseract OCR Engine,” in *Proceedings of the Ninth International Conference on Document Analysis and Recognition*, 2007.
2. G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
3. S. Brin and L. Page, “The Anatomy of a Large-Scale Hypertextual Web Search Engine,” *Computer Networks and ISDN Systems*, vol. 30, pp. 107–117, 1998.
4. OWASP Foundation, “OWASP Application Security Verification Standard (ASVS),” OWASP Foundation.
5. FastAPI Documentation, “FastAPI: Modern Web Framework for Building APIs with Python.”
6. React Documentation, “React: A JavaScript Library for Building User Interfaces.”